Python Fundamentals

- Introduction to Python

1. Introduction to Python and its Features (simple, high-level, interpreted language)
    a. Python is a popular programming language known for being simple, readable, and powerful.
    b. It was created by Guido van Rossum and first released in 1991.
    c. Today, it is widely used in web development, data science, artificial intelligence, automation, and more.
    d. Python is designed to be easy for beginners while still being strong enough for professional developers.

➔ Features of Python :

◆ **Simple & easy to learn**

- Python's syntax is clean and close to plain English, making it beginner-friendly.
- Syntax:

```
print("Hello, World!")
```

- Compare that to Java, where you'd need a class, a main method, and multiple lines just to do the same thing. Python removes all that ceremony.

◆ **High-Level Language**

- Python is a high-level language, meaning it abstracts away low-level details like memory management, pointers, and hardware interaction. You focus on what to do, not how the computer does it.
- It handles complex tasks like memory allocation automatically.

◆ **Interpreted Language**

- Python code is executed line by line by an interpreter, rather than being compiled into machine code beforehand. This means:
    - No need to compile before running.
    - Easier to debug errors
    - Faster development & testing
    - Errors are caught at runtime, one at a time

◆ **Dynamically Typed**

- You don't need to declare variable types. Python figures it out automatically.
- Ex:

```
pythonx = 10   integer
x = "hello"   # string
x = 3.14      # float
```

- **Cross Platform**
  - Python runs on Windows, macOS, Linux, and more — write once, run anywhere.

- **Object-Oriented**
  - Python supports object-oriented programming (OOP):
    - Classes
    - Objects
    - Inheritance
    - Polymorphism

- **Large Standard Library**
  - Python comes with a rich set of built-in modules for:
    - File handling
    - Math operations
    - Web services
    - Data processing

- **Open Source and Free**
  - Python is free to use.
  - Supported by a large global community.

- **Extensible and Embeddable**
  - Can be integrated with languages like C/C++.
  - Can be embedded into other applications
  .
- **Used Everywhere**
  - Web Development (Django, Flesk)
  - Data Science (Pandas, Numpy)
  - Machine Learning (TensorFlow, PyTorch)
  - Automation (Selenium, PyAutoGUI
  - Cybersecurity (Scapy, Nmap Scripting)

2. History and evolution of Python.
   a. Python was created by Guido van Rossum, a Dutch programmer. He was famously given the title "Benevolent Dictator For Life" (BDFL) by the Python community — meaning he had final say on all language decisions (he stepped down from this role in 2018).
   b. The name "Python" didn't come from the snake — it was inspired by the British comedy series "Monty Python's Flying Circus", which van Rossum was a fan of.

➔ **History & Evolution**

Year	Event
1989	Guido van Rossum started developing Python

1991	Python 0.9.0 first published
1994	Python 1.0 officially released
2000	Python 2.0 — added list comprehensions, garbage collection
2008	Python 3.0 — major redesign, not backward compatible
2020	Python 2 retired; Python 3 fully adopted
2024	Python ranked **#1** most popular language (TIOBE)

3. Advantages of using Python over other programming languages.
    a. **Simple and Readable Syntax**
        i. Python code reads like plain English, making it easier to write and understand compared to languages like C++ or Java.

    b. **Easy to Learn**
        i. Beginner-friendly — minimal syntax rules
        ii. Less boilerplate code
        iii. Ideal for students, scientists, and non-programmers

    c. **Free and Open Source**
        i. Completely free to download and use
        ii. Large open-source community constantly improving it
        iii. Thousands of free libraries available

    d. **Large Standard Library**
        i. Python comes with a massive built-in library — no need to write everything from scratch.
        ii. Built-in Lib :
            1. import math # Mathematics
            2. import os # File system
            3. import datetime # Date & Time
            4. import json # JSON handling
            5. import random # Random numbers

    e. **Cross-Platform**
        i. Write code once, run it anywhere
        ii. Works on Windows, macOS, Linux
        iii. No changes needed when switching OS

    f. **Dynamically Typed**
        i. No need to declare variable types — Python handles it automatically, saving time.
        ii. Ex .

```
x = 10          # int
```

```
x = "Python"    # string — no error!
x = 3.14        # float — still fine!
```

g. **Versatile — One Language, Many Domains**

Domain	Python Usage
Web Development	Django, Flask
Data Science	Pandas, NumPy
Machine Learning	TensorFlow, PyTorch
Automation	Selenium, PyAutoGUI
CyberSecurity	Scapy, Nmap
Game Development	Pygame

h. **Interpreted Language**
   i. No compilation step — run code instantly
   ii. Errors are easy to find and fix
   iii. Great for rapid prototyping

i. **Strong Community Support**
   i. Millions of developers worldwide
   ii. Countless tutorials, forums, and documentation
   iii. Active support on Stack Overflow, GitHub, Reddit

j. **Integration Friendly**
   i. Python easily integrates with other languages and technologies.

```
# Connect to databases, APIs, C/C++ libraries, Java, and more
import sqlite3       # Database
import requests      # Web APIs
```

4. Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).

5. Writing and executing your first Python program.

● Programming Style

1. Understanding Python's PEP 8 guidelines.
   a. PEP 8 (Python Enhancement Proposal 8) is the official style guide for writing code in Python. It provides conventions that help developers write readable, consistent, and maintainable code.

➔ **Code Layout**

- PEP 8 recommends using 4 spaces for indentation to ensure consistency. The maximum line length should be 79 characters to improve readability. Blank lines are used to separate functions and classes for better structure

➔ **Naming Conventions**
- Different elements in Python follow specific naming styles:
  - Variables and functions use lowercase_with_underscores
  - Class names use CapitalizedWords (PascalCase)
  - Constants are written in UPPERCASE_WITH_UNDERSCORES
- These conventions make code easier to understand at a glance.

➔ **Imports**
- Imports should be placed at the top of the file and grouped into:
  - Standard library imports
  - Third-party libraries
  - Local modules
- This organization improves clarity and avoids confusion.

➔ **Whitespace Usage**
- Proper spacing improves readability:
  - Use spaces around operators (x = a + b)
  - Avoid unnecessary spaces inside brackets or parentheses
  - Separate functions and classes using blank lines

➔ **Comments and Documentation**
- Comments should clearly explain the purpose of the code.
- Docstrings are used to describe functions, classes, and modules.
  Example:
  def add(a, b):
  """Returns the sum of two numbers."""
  return a + b

➔ **Writing Clean Expressions**
- PEP 8 encourages simple and readable expressions.
- Avoid overly complex code and prefer clarity over brevity.

➔ **Consistency**
- The most important rule in PEP 8 is consistency. Following the same style throughout a project makes the code easier to read and maintain.

2. Indentation, comments, and naming conventions in Python.

➔ **Indentation**
- Indentation refers to the spaces at the beginning of a line of code. In Python, indentation is not just for readability—it is mandatory and defines code blocks.

- Use 4 spaces per indentation level
- Do not mix tabs and spaces
- Proper indentation improves readability and avoids errors

Ex :

```
def greet():
    if True:
        print("Hello, World!")
```

➔ **Comments**

- ◆ Comments are used to explain code and make it easier to understand. They are ignored by the Python interpreter.
- ◆ Types of Comments:

❖ Single-line comments start with #

```
# This is a single-line comment
```

❖ Multi-line comments can be written using multiple # or triple quotes

```
"""
This is a multi-line comment
"""
```

❖ Docstrings are used to describe functions, classes, or modules

```
def add(a, b):
    """Returns the sum of two numbers."
    return a + b
```

➔ **Naming Conversions**

- ◆ Naming conventions make code consistent and easy to read.
- ❖ **Variables and functions** : lowercase_with_underscores

  Ex :

  total_marks = 90

- ❖ **Class names** : CapitalizedWords (PascalCase)

  Ex :

  class StudentRecord:

  pass
- ❖ **Constants** : UPPERCASE_WITH_UNDERSCORES

  Ex :

  MAX_LIMIT = 100

- ❖ **Private variables** : start with a single underscore _variable

3. Writing readable and maintainable code.
    a. Writing readable and maintainable code is an essential skill in Python programming. It ensures that code is easy to understand, debug, and modify by others or even by the original developer in the future.

➔ **Use Meaningful Names**
- ◆ Choose clear and descriptive names for variables, functions, and classes.
- ◆ Meaningful names improve understanding of the code's purpose.

➔ **Follow Consistent Style (PEP 8)**
- ◆ Consistency makes code easier to read and maintain.
- ◆ Adhere to the guidelines of PEP 8:
    - Use 4 spaces for indentation
    - Keep line length reasonable
    - Use proper spacing and formatting

➔ **Keep Code Simple and Clear**
- ◆ Avoid overly complex logic. Break large problems into smaller functions.
    - Simple and readable code is better than clever but confusing code.

➔ **Use Comments and Docstrings**
- ◆ Explain the purpose of code where necessary.
- ◆ Write comments only when needed
- ◆ Avoid redundant or obvious comments

➔ **Modular Programming**
- ◆ Divide code into smaller functions and modules:
- ◆ Each function should perform one task
- ◆ Reusable code reduces duplication

➔ **Avoid Code Duplication**
- ◆ Reuse existing functions instead of repeating code. This makes updates easier and reduces errors.

➔ **Proper Error Handling**
- ◆ Handle errors using exceptions to prevent crashes and improve reliability.

➔ **Keep Code Organized**
- ◆ Group related code together
- ◆ Maintain a logical structure

- Use proper file and folder organization

- Core Python Concepts

1. Understanding data types: integers, floats,strings, lists, tuples, dictionaries,sets.
    a. In Python, data types define the type of value a variable can store. Python provides several built-in data types to handle different kinds of data.

➔ **Integers (int) :**
- Integers are whole numbers, which can be positive or negative, without decimal points.

➔ **Floats (float) :**
- Floats represent decimal or floating-point numbers.

➔ **Strings (str) :**
- Strings are sequences of characters enclosed in single (' ') or double (" ") quotes.

➔ **Lists (list) :**
- Lists are ordered and mutable (changeable) collections of elements.
- Lists are created using square " **[ ]** " brackets.

➔ **Tuples (tuple) :**
- Tuples are ordered but immutable (cannot be changed after creation)
- Tuples are created using round " **( )** " brackets.

.

➔ **Dictionaries (dict) :**
- Dictionaries store data in key-value pairs.
- Dictionaries are created using curly " **{ }** " brackets.

➔ **Sets (set) :**
- Sets are unordered collections of unique elements (no duplicates allowed).
- Sets are created using round " **( )** " brackets.

2. Python variables and memory allocation.
    a. In Python, variables are used to store data, and memory allocation is handled automatically by the system.

➔ **Variables in Python**
- A variable is a name given to a value or object.
    - Python is dynamically typed, so you don't need to declare the type explicitly.
    - A variable can hold different types of values at different times.
      Ex :
      x = 10 # integer
      x = "Hello" # now a string

➔ **Variable Assignment**

- ◆ When you assign a value to a variable
  - Creates an object in memory
  - Assigns the variable name to refer to that object

  Ex :
  a = 5
  b = a

➔ **Memory Allocation in Python**

- ◆ Memory is managed automatically by Python (no need for manual allocation like in C/C++).
- ◆ Python uses dynamic memory allocation, meaning memory is allocated at runtime.
- ◆ Variables store references (addresses) to objects, not the actual data.
- ◆ When a variable is reassigned, it points to a new object.

  Ex :
  x = 10
  x = 20

➔ **Garbage Collection**

- ◆ Python automatically frees unused memory using garbage collection.
  - If no variable refers to an object, it becomes eligible for deletion.
  - This improves memory efficiency.

➔ **Object Identity and id()**

- ◆ Each object has a unique identity in memory.

  Ex :
  x = 10
  print(id(x))

3. Python operators: arithmetic, comparison, logical, bitwise.
   a. In Python, operators are symbols used to perform operations on variables and values. Python provides different types of operators for various purposes.

➔ **Arithmetic Operators**

- ◆ These are used to perform mathematical calculations.

Operator	Description	Example
+	Addition	A + B
-	Subtraction	A - B
*	Multiplication	A * B
/	Divison	A / B
%	Modules	A % B

**	Exponent (Power)	A ** B
//	Floor Divison	A // B

➔ **Comparison Operators**

- ◆ These operators compare two values and return either True or False.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

➔ **Logical Operators**

- ◆ Logical operators are used to combine conditional statements.

Operator	Meaning
and	Return true if both conditions are true
or	Return true if at least one condition is true
not	Reverses the result

➔ **Bitwise Operators**

- ◆ These operators work on binary (bit-level) values.

Operator	Meaning
&	AND
`	`
^	XOR
~	NOT
<<	Left shift
>>	Right shift

- Conditional Statements

1. Introduction to conditional statements: if, else, elif.
    a. Conditional statements let your program make decisions — executing different code based on whether conditions are True or False.

➔ **Simple If :**

- The if statement is used to test a condition. If the condition is true, the block of code is executed.

Syntax :

If condition :

Statement

Example **:**

```
age = 18

if age >= 18:
    print("You are eligible to vote")
```

➔ **If Else :**

- The else statement is used when the if condition is false.

Syntax :

If condition :

Statement

Else :

Statement

Example :

```
a = int(input("Enter any Number : "))

if a % 2 == 0:
    print("Number is Even")
else:
    print("Number is Odd")
```

:

➔ **Nested If :**

- A nested if is an if statement inside another if statement. It is used when one condition depends on another condition.

Syntax :

If condition :

If condition :
Statement
Else :
Statement
Else :
Statementt

Example :

```
# find max number from 3 numbers
a=int(input("Enter A : "))
b=int(input("Enter B : "))
c=int(input("Enter C : "))

if a>b:
    if a>c:
        print("A Is Max")
    else:
        print("C Is Max")
elif b>c:
    print("B Is Max")
else:
    print("C Is Max")
```

➔ **Leader if :**

- Used when multiple conditions need to be checked.

Syntax :

If condition :
Statement
elif condition :
Statement
Else :
Statement

Example :

```
marks = 80

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

:

- Looping (for, while)

1. Introduction to for and while loops.
    a. Loops are used in Python to repeat a block of code multiple times.

➔ for loop

- A for loop is used when we know how many times we want to repeat something.
  Syntax :
  for variable in sequence:
  # code

  Ex :

```
# List of fruits
List1 = ['apple', 'banana', 'mango']

# Using for loop to print each fruit
for fruit in List1:
    print(fruit)
```

➔ While loop

- A while loop is used when the number of repetitions is not fixed.
- It runs as long as the condition is True.
  Syntax :
  while condition:
  # code

  Ex :

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

2. How loops work in Python.
    a. Loops execute a block of code repeatedly until a condition becomes false or the sequence ends.

➔ Working of a for Loop

- Python takes the first item from the sequence.
- Executes the code block.
- Moves to the next item.
- Continues until all items are finished

➔ Working of a while Loop
- ◆ Python checks the condition.
- ◆ If the condition is True, the code runs.
- ◆ After execution, the condition is checked again.
- ◆ The loop stops when the condition becomes False

.

3. Using loops with collections (lists, tuples, etc.)
   a. Loops are commonly used to access elements from collections like lists, tuples, strings, and dictionaries.

❖ Using Loop with List

```
numbers = [1, 2, 3, 4]

for num in numbers:
    print(num)
```

❖ Using Loop with Tuple

```
colors = ("red", "green", "blue")

for color in colors:
    print(color)
```

❖ Using Loop with Dictionary

```
student = {
    "name": "Niya",
    "age": 26
}

for key, value in student.items():
    print(key, ":", value)
```

❖ .Using Loop with String

```
word = "Python"

for letter in word:
    print(letter)
```

- Generators and Iterators

1. Understanding how generators work in Python
    a. Generators are special functions in Python that produce values one at a time instead of returning all values at once.
    b. They use the yield keyword.
    c. Generators are memory-efficient because they generate values only when needed.

➢ How Generators Work

- When the generator function is called, it does not execute immediately.
- It returns a generator object.
- The next() function starts execution until a yield statement is found.
- The value is returned, and the function pauses.
- Execution resumes from the same point on the next next() call.

.

2. Difference between yield and return.

Feature	yield	return
Purpose	Produces values one at a time	Ends a function and sends back a value
Function Type	Generator function	Normal function
Execution	Function pauses and resumes later	Function stops completely
Memory Usage	Generates data lazily (memory efficient)	Stores all data at once
Keyword Used In	Generators	Regular functions
Iteration Support	Can be iterated using loops	No automatic iteration

3. Understanding iterators and creating custom iterators.
    a. An **iterator** is an object that gives values one by one.
    b. It uses:
        i. iter() → creates iterator
        ii. next() → gets next value

Ex

```
numbers = [1, 2, 3]

it = iter(numbers)

print(next(it))
print(next(it))
print(next(it))
```

➔ Custom Iterator :

Ex :

```
class Counter:
    def __init__(self, limit):
        self.limit = limit
        self.num = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.num <= self.limit:
            value = self.num
            self.num += 1
            return value
        else:
            raise StopIteration

c = Counter(5)

for i in c:
    print(i)
```

:

- Functions and Method

1. Defining and calling functions in Python.
    a. A function is a reusable block of code that performs a specific task.

➔ **Defining a function :**

```
def greet():
    print("Hello, World!")
```

➔ **Calling a function :**

```
def greet():
    print("Hello, World!")

greet()
```

2. Function arguments (positional, keyword, default).
    a. Python supports several types of arguments.

➔ **Positional Arguments :**

- Arguments are assigned according to their position.

```
def introduce(name, age):
    print(name, age)

introduce("John", 25)
```

➔ **Keyword Arguments :**

- Arguments are specified using parameter names.

```
def introduce(name, age):
    print(name, age)

introduce(age=25, name="John")
```

➔ **Default Arguments :**

- A parameter can have a default value.

```
def greet(name="Guest"):
    print("Hello,", name)

greet()
greet("Alice")
```

3. Scope of variables in Python.
    a. Scope determines where a variable can be accessed.

➔ **Local Scope**

- Variables declared inside a function are local.

```
def show():
    x = 10
    print(x)

show()
```

- X exists only inside show().

➔ **Global Scope**

- Variables declared outside functions are global.

```
x = 20

def show():
    print(x)

show()
```

➔ **Modifying Global Variables**

- Use the global keyword.

```
count = 4

def increase():
    global count
    count += 1

increase()
print(count)
```

4. Built-in methods for strings, lists, etc.

➔ **String functions :**

```
text = "Niyati Patel 2699"

print("Length of text is : ", len(text))
print("Capitalize text :", text.capitalize())
print("Casefold Text : ", text.casefold())
print("Upper Text : ", text.upper())
print("Lower Text : ", text.lower())
print("Word Count : ", text.count("i"))
print(text.endswith("26"))
print(text.startswith("Niyati"))
print(text.find("Patel"))
print(text.index("i", 2))
```

➔ **List functions :**

```
list = [1,5,3,True, 1.1, "Niya", "ABC", 100, 200, 2.9, False, 10, 20]

print("Orignal List : ",list)
list.append(200)
print("Append 200 at last index in list : ",list)
#list.clear()
#print(list)
list1 = list.copy()
print("Copy of original list : ",list1)
list1.append(1000)
print("Append 1000 in last index at list1 : ",list1)
list2 = list
list2.append(300)
print(list)
print(list2)
print("Count 1 in list ",list.count(1))
#count is 2 becuase 1 itself and 2 is True
#true = 1, flase = 0
list3 = [2000,2200,4569]
print("List 3 : ",list3)
list.extend(list3)
```

➔ **Dictionary functions :**

```
d = {101:"Diya",345:"Bhavika",909:"Niya",566:"Om",234:"Pralaj",787:"Monik",454:"Parth"}

print(d)
print(d[909])
print(d.get(101))
print(d.items())
print(d.keys())
print(d.values())
print(d.pop(566))
print(d)
d.popitem()
print(d)
d1 = {454:"Parth",566 : "Om"}
d.update(d1)
print(d)
d[202] = "Ravi"
print(d)
```

- Control Statements (Break, Continue, Pass)

1. Understanding the role of break, continue, and pass in Python loops.
    a. In Python, break, continue, and pass are control statements that affect how loops behave, but they serve different purposes.

➔ **Break - Exit lloop immediately :**

- The break statement stops the loop as soon as it is encountered.
  Ex :

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
#output 1,2
```

➔ **Continue - skip the current iteration**

- The continue statement skips the rest of the current iteration and moves to the next one.
  Ex :

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)

#output 1,2,4,5
```

➔ **Pass - Do nothing**

- The pass statement is a placeholder that performs no action.
  Ex :

```
for i in range(1, 6):
    if i == 3:
        pass
    print(i)

#output : 1,2,3,4,5
```

- String Manipulation

1. Understanding how to access and manipulate strings.
   a. Strings are ordered sequences of characters. Each character has an index, starting at 0 from the left and -1 from the right.
   b. Strings are immutable — you cannot change a character in place (s[0] = 'J' raises an error). Any "modification" creates a new string.

2. Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).

➔ Concatenation

- ◆ Joining strings with “+”

  Ex :

```
first = "Hello"
second = "World"
result = first + " " + second

# "Hello World"
```

➔ Repetition

- ◆ Repeating with *

  Ex :

```
print("-" * 10)

# "----------"
```

➔ Common string methods

```
text = "Niyati Patel 2699"

print("Length of text is : ", len(text))
print("Capitalize text :", text.capitalize())
print("Casefold Text : ", text.casefold())
print("Upper Text : ", text.upper())
print("Lower Text : ", text.lower())
print("Word Count : ", text.count("i"))
print(text.endswith("26"))
print(text.startswith("Niyati"))
print(text.find("Patel"))
print(text.index("i", 2))
print(text.isalnum())
print("niya".isalpha())
print("2699".isnumeric())
print(text.isspace())
```

3. String slicing.
    a. Syntax: s[start:stop:step] — start is inclusive, stop is exclusive.

```
s = "Hello, World Python Example"
print(s[3:13])   #lo, World
print(s[:15])    #Hello, World Py
print(s[2:])     #llo, World Python Example
print(s[1:15:4]) #e,rP
print(s[::3])    #Hl r tnxp
```

- Advanced Python (map(), reduce(), filter(), Closures and Decorators)

1. How functional programming works in Python.
    a. Python isn't a purely functional language, but it supports functional-style programming: treating functions as first-class objects (can be assigned to variables, passed as arguments, returned from other functions) and favoring pure functions (no side effects, output depends only on input) over mutable state.

    Ex :

```
def square(x):
    return x * x

operation = square          # assign function to a variable
print(operation(5))         # 25

def apply(func, value):     # pass function as argument
    return func(value)

print(apply(square, 6))     # 36
```

2. Using map(), reduce(), and filter() functions for processing data.

➔ **map(func, iterable)**
- ◆ applies func to every element, returns a map object

Ex :

```
nums = [1, 2, 3, 4]
squared = map(lambda x: x ** 2, nums)
print(list(squared))   # [1, 4, 9, 16]
```

➔ **filter(func, iterable)**

- Keeps elements where func returns true

  Ex :

```
nums = [1, 2, 3, 4, 5, 6]
evens = filter(lambda x: x % 2 == 0, nums)
print(list(evens))    # [2, 4, 6]
```

➔ **reduce(func, iterable)**

- combines all elements into a single value (needs from functools import reduce)

  Ex :

```
from functools import reduce

nums = [1, 2, 3, 4]
total = reduce(lambda a, b: a + b, nums)
print(total)    # 10

product = reduce(lambda a, b: a * b, nums)
print(product)    # 24
```

3. Introduction to closures and decorators.

➔ **Closures :**

- an inner function that remembers variables from its enclosing scope, even after the outer function has finished running

  Ex :

```
def make_multiplier(n):
    def multiplier(x):
        return x * n
# 'n' is captured from outer scope
    return multiplier

times3 = make_multiplier(3)
print(times3(10))    # 30

times5 = make_multiplier(5)
print(times5(10))    # 50
```

➔ **Decorators :**

- functions that wrap another function to extend/modify its behavior without changing its code. Built using closures and the @ syntax.
  Ex :

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before the function runs")
        result = func(*args, **kwargs)
        print("After the function runs")
        return result
    return wrapper

@my_decorator
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
```